A COURSE BUILT FROM THIS REPOSITORY

Learn React and Next.js by building Seltra

Simple enough to read like a story. Precise enough to survive a technical interview. Every idea is tied to a real file in the Seltra frontend.

Project: seltra-web frontend · React 18.3 · Next.js 16 App Router · TypeScript
Stack you will meet: components, hooks, context, server components, routing, data fetching, forms, auth.

Study order: read a chapter, open the file it names, then answer the interview question out loud in under 90 seconds. If you can point at the file while you talk, you understand it.

How to use this course

You do not need to memorize the whole Seltra dashboard. You need a small set of principles, and you need to see those principles in working code. This document does both.

Each idea has three layers:

1. **Like you are five.** A picture you can hold in your head.
2. **In this project.** The exact Seltra file where that picture is real.
3. **Say this in the interview.** A short answer a hiring manager can follow.

Read it twice. The first time, only the kid boxes and the project boxes. The second time, the interview boxes. Then rebuild the tiny versions in the practice chapter without looking.

> **LIKE YOU ARE FIVE**
>
> Seltra is a toy shop that builds other toy shops. The website is a book. Each web address is a page in the book. React is the set of stickers and moving parts on the page. Next.js is the printing press that decides which page to print, and whether the printing happens in the back room or in your hands.

0. The map of this application

There are two programs. You are learning the front one.

Piece	Where	Job
Frontend	`seltra-web/frontend`	What people see and click. React + Next.js.
Backend	`seltra-web/backend`	The office that stores users, stores, products, and emails. The browser asks it questions.
Database / login	Supabase	A rented filing cabinet and a lock on the door.

The frontend talks to the backend with `fetch`. A public store page can ask the server to fetch, so the secret server address stays off the browser. A signup form runs in the browser, so it uses a public address.

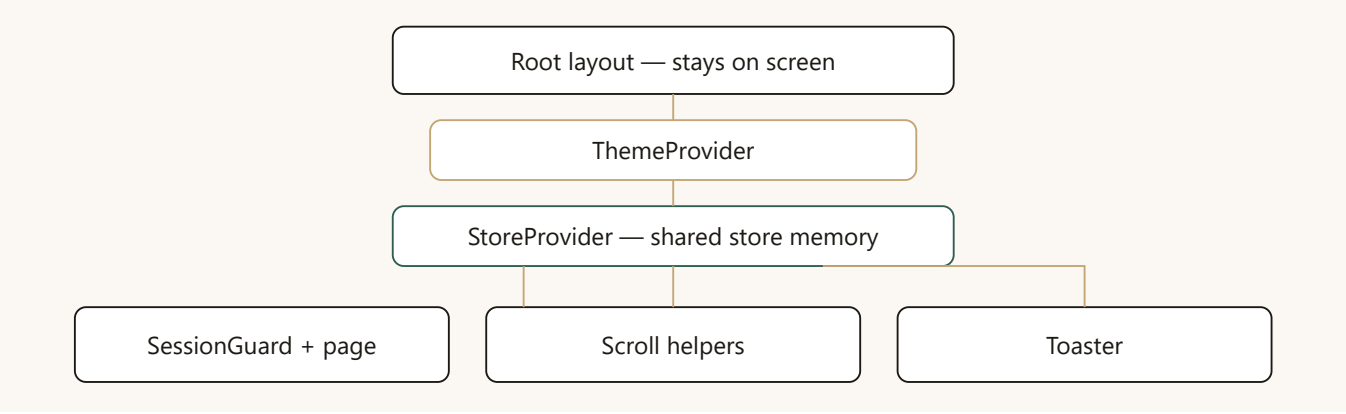


Figure 1. The browser only talks to the Next.js app. That app asks the Nest API and Supabase for the real records.

The folders that matter

Path	Meaning
`frontend/app`	URLs. A folder is a path. `page.tsx` is the screen. `layout.tsx` is the frame around screens.
`frontend/components`	Reusable pieces: buttons, header, signup, storefront.
`frontend/context`	Shared memory. Today: which store is active.
`frontend/lib`	Helpers that are not screens: class names, Supabase, SEO text.
`frontend/hooks`	Reusable React behavior, such as toasts.
`frontend/public` and assets	Files the browser can request directly, like logos.

`tsconfig.json` says `"@/*": ["./*"]`. That is why imports look like `@/components/ui/button` instead of `../../components/ui/button`. The `@` is the frontend folder.



The frame in `app/layout.tsx`. Only the page inside SessionGuard swaps when the URL changes.

> **SAY THIS IN THE INTERVIEW**
>
> "Seltra's UI is a Next.js App Router app. Routes live in `app`. Interactive UI lives in client components. The root layout wraps every page with theme, store context, and a session guard. Public store pages are async server components that fetch by slug and revalidate every 30 seconds. Forms and the dashboard are client components because they need state and browser APIs."

1. JavaScript you need before React

React is JavaScript with a drawing language. If these six ideas are solid, React stops feeling magical.

1.1 A function is a machine

```
function greet(name) {
  return "Hello " + name
}
greet("Ama") // "Hello Ama"
```

A React component is this kind of machine. You give it inputs. It returns a description of UI.

1.2 Objects and arrays

```
const store = { name: "De Yogo", slug: "de-yogo" }
store.name // "De Yogo"

const products = [{ id: "1", price: 49 }, { id: "2", price: 28 }]
products.map((product) => product.price) // [49, 28]
```

You will map arrays into lists of components. You will read fields off store objects. That is most of the dashboard.

1.3 Copy, do not scribble

```
const next = { ...store, name: "New name" }
setData((current) => ({ ...current, email: "a@b.com" }))
```

The three dots copy the old object and then change one field. React notices a new object. If you change the old object in place, React may not notice.

> **IN THIS PROJECT**
>
> `SignupFlow.tsx` updates the form like this: `setData((current) => ({ ...current, [key]: value }))`. That is the correct pattern. The function form of `setData` means "start from the latest copy," which matters if two updates happen close together.

1.4 Destructuring and rest

```
const { className, variant, size, asChild = false, ...props } = incoming
```

Pull out the fields you care about. `...props` is "everything else," passed on to the real button. See `components/ui/button.tsx`.

1.5 Async and fetch

```
const res = await fetch(url)
if (!res.ok) return null
return res.json()
```

`await` means "pause this function until the answer arrives, but do not freeze the whole program." `getStore` in `app/store/[slug]/page.tsx` is exactly this.

1.6 Modules

`export` puts a name on the shelf. `import` takes it off the shelf. A default export is the main thing in the file. A named export is one of several. `page.tsx` default-exports the page component and also named-exports `metadata`.

> **SAY THIS IN THE INTERVIEW**
>
> "React state should be updated immutably. I copy objects with the spread operator and use the functional updater when the next value depends on the previous one. Lists are rendered with `map`. Server data is loaded with async `fetch`."

---

2. React, from a blank page

2.1 A component

> **LIKE YOU ARE FIVE**
>
> A component is a cookie cutter. The cutter is the function. Each cookie is what you see on the screen. You can stamp the same cutter many times with different icing.



Figure 2. One component function, many copies on the screen. The icing is the props: same cutter, different name or price.

```
function Hello({ name }: { name: string }) {
  return <p>Hello {name}</p>
}
```

Rules that interviewers listen for:

- The function name starts with a capital letter. `Hello` is a component. `hello` is just a function, and JSX will not treat it as a component.
- It returns JSX, or `null`.
- It must be pure during render: same inputs, same output, no surprises like writing to a database while drawing.
- Surprises (timers, network, subscriptions) belong in effects or in event handlers, not in the body of render.

2.2 JSX

> **LIKE YOU ARE FIVE**
>
> JSX is a letter to the browser written in a mix of HTML and JavaScript. Curly braces mean “stop writing the letter and do some math.”

```
const price = 49
return <p className="price">GHS {price}</p>
```

HTML habit	JSX habit	Why
`class`	`className`	`class` is a JavaScript word.
`for`	`htmlFor`	Same reason. `for` is a loop.
`onclick`	`onClick`	React events are camelCase.
close optional tags	`<img />`, `<input />`	JSX must be valid XML-ish.
many roots sometimes	one parent, or `<>...</>`	A component returns one tree.

This project sets `"jsx": "react-jsx"` in `tsconfig.json`. You do not write `import React from "react"` just to use JSX. The compiler adds the runtime. You still import hooks by name: `import { useState } from "react"`.

2.3 Props: stickers on the way in

> **LIKE YOU ARE FIVE**
>
> Props are instructions from the parent. The child may read them. The child does not edit them. If the child wants a change, it asks the parent by calling a function the parent handed down.

```
export function SignupFlow({ formPhoto, donePhoto }: { formPhoto: string | null; donePhoto: string | null }) {
  // formPhoto and donePhoto are props
}
```

Props are read-only. They can be strings, numbers, objects, arrays, functions, or other components (`children`).

`children` is the special prop for whatever you put between the opening and closing tags.

```
function Card({ children }: { children: React.ReactNode }) {
  return <section className="card">{children}</section>
}
// <Card><p>Hi</p></Card>
```

> **IN THIS PROJECT**
>
> `RootLayout` receives `children` and wraps them in theme, store, and session providers. `SessionGuard` receives `children` and returns them after setting up logout timers. `StoreProvider` does the same. This is composition: frames wrap pictures without the picture knowing about the frame.

2.4 One-way data flow

Data moves down through props. Events move up through callback props. There is no secret tunnel where a child mutates a parent's variable.



Figure 3. Props travel down. A click travels back up as a function call. The child never edits the parent's notebook.

```
function Parent() {
  const [open, setOpen] = useState(false)
  return <Child open={open} onClose={() => setOpen(false)} />
}
```

> **SAY THIS IN THE INTERVIEW**
>
> "React data flow is unidirectional. Parents pass state down as props. Children request changes through callbacks. That makes the UI predictable: I can find who owns a value by walking up the tree. When many distant children need the same value, I lift it into context instead of threading props through components that do not care."

2.5 State: the component's notebook

> **LIKE YOU ARE FIVE**
>
> Props are what you were given. State is what you remember. When the notebook changes, React redraws that component.

```
const [step, setStep] = useState(0)
const [loading, setLoading] = useState(false)
```

Calling `setStep(1)` does two things: it stores `1`, and it schedules a new render. You do not push pixels yourself.

Important rules:

- State updates are asynchronous. Right after `setStep(1)`, `step` is still the old number until the next render.
- If the new value depends on the old one, use `setStep((current) => current + 1)`.
- Do not store what you can calculate. Seltra's form errors are calculated with `useMemo`, not stored as a second copy of the truth.
- Many `useState` calls are fine when the fields are independent. If many fields always change together through named actions, `useReducer` is clearer.

> **IN THIS PROJECT**
>
> `SiteHeader.tsx` keeps `open`, `scrolled`, and `authOpen` in state. Those are real memories: menu open or closed, whether the user has scrolled, whether the login modal is showing. `SignupFlow` keeps the wizard step and the form fields. The error object is derived from those fields, which is the right split.

2.6 Rendering, in one paragraph you can repeat

> **SAY THIS IN THE INTERVIEW**
>
> "Render means calling the component function to get a React element tree. React compares that tree to the previous one and updates the DOM where they differ. A state or prop change schedules a render of that component and its children. Render itself should be pure. Side effects go in event handlers or `useEffect`. I never change state during render unless it is the narrow pattern React documents for adjusting state when a prop changes, and even then I prefer deriving the value."

People say "virtual DOM." The accurate version: React keeps a description of the UI in memory, diffs it, and patches the real DOM. You do not need to claim React is fast because of a virtual DOM. You say React skips DOM writes when the description did not change, and it uses keys so lists can be matched correctly.

2.7 Lists and keys

> **LIKE YOU ARE FIVE**
>
> If you have three toy cars in a row and you remove the middle one, name tags stop you from repainting the wrong car. The key is the name tag.



Figure 4. The key is the name tag. React uses it to keep the right state on the right row when the list changes.

```
{products.map((product) => (
  <li key={product.id}>{product.name}</li>
))}
```

- Keys must be stable and unique among siblings.
- Use an id from the data. Do not use the array index if the list can be reordered, filtered, or inserted into.
- Index keys make input state stick to the wrong row after a reorder. That is the classic interview bug.
- Keys are not props. The child does not receive `key` unless you also pass it under another name.

2.8 Conditional UI

```
{loading ? <Spinner /> : <button onClick={submit}>Submit</button>}
{error ? <p>{error}</p> : null}
{done && <Success />}
```

`done && <Success />` is fine when `done` is a real boolean. If the value can be `0` or `""`, React will print that value. Prefer a ternary when you are not sure.

2.9 Events

```
<button type="button" onClick={goNext}>Next</button>
<input value={email} onChange={(event) => setEmail(event.target.value)} />
```

Pass the function. Do not call it. `onClick={goNext}` waits for the click. `onClick={goNext()}` runs during render. If you need an argument, wrap it: `onClick={() => remove(id)}`.

In forms, `onSubmit` on the `<form>` and `event.preventDefault()` stop the browser from doing a full page reload.

2.10 Controlled inputs

> **LIKE YOU ARE FIVE**
>
> A controlled input is a puppet. React holds the string. The input only displays it. Every keystroke tells React the new string, and React writes it back.

An uncontrolled input keeps the string inside the DOM. You read it later with a ref. Controlled is what Seltra's signup and auth forms do, because validation needs the value on every change.

> **SAY THIS IN THE INTERVIEW**
>
> "A controlled input's value lives in React state. The DOM is a view of that state. I use it when I need to validate, disable a button, or format as the user types. An uncontrolled input is fine for a throwaway field I only read on submit. I do not switch a component from uncontrolled to controlled by starting with `value={undefined}` and later passing a string."

---

3. Hooks, the ones this app actually uses

> **LIKE YOU ARE FIVE**
>
> Hooks are special tools you may only use at the top of a component, in the same order every time. You do not put them inside `if`, loops, or nested functions. React recognizes each hook by its place in line, not by its variable name.

> **TRAP**
>
> If the first render calls three hooks and the second render calls two, React pairs them wrong and state leaks between variables. That is why the rules of hooks exist. Interviewers love this question. Answer: hooks are stored in a list by call order, so the order must be stable.

3.1 useState

Covered above. Lazy initial state matters in this repo:

```
const [store, setStoreState] = useState<StoreData | null>(() => read())
```

`useState(read())` would call `read()` on every render and throw away the result. `useState(() => read())` calls it once. `read()` also checks `typeof window === "undefined"` because the server has no `localStorage`.

3.2 useEffect

LIKE YOU ARE FIVE

Render is drawing the picture. An effect is what you do after the picture is on the wall: start a timer, listen for scrolling, subscribe to login changes. The function you return is cleaning up before you draw again or leave the room.

```
useEffect(() => {
  const onScroll = () => setScrolled(window.scrollY > 8)
  window.addEventListener("scroll", onScroll, { passive: true })
  return () => window.removeEventListener("scroll", onScroll)
}, [])
```

The dependency array is the list of values the effect reads from React.

- `[]` means "after the first paint on the client, and clean up when the component unmounts."
- `[pathname, router]` means "also re-run when the path changes."
- Missing a dependency is a stale closure: the effect keeps seeing an old `pathname`.
- Putting an object you create during render into the array re-runs the effect every time, because it is a new object every render.

IN THIS PROJECT

`SessionGuard.tsx` is the textbook effect. On mount and whenever `pathname` changes, it subscribes to Supabase auth, schedules an idle logout, and listens for activity, storage, and a custom `seltra:auth-expired` event. The cleanup clears the timer, unsubscribes, and removes listeners. If that cleanup were missing, every navigation would stack another set of listeners.

SAY THIS IN THE INTERVIEW

"I use `useEffect` to synchronize with something outside React: the DOM, timers, subscriptions, browser storage events. I return a cleanup. I do not use an effect to compute values I could derive during render, and I do not use an effect to fetch when a server component or a data library can own that request. Event handlers are better than effects for work caused by a click."

TRAP

Fetching in `useEffect` causes a loading flash and a second request after hydration. In the App Router, if the page can be a server component, fetch there. The store page does this. The dashboard cannot, because it is a large client island, so client fetches are a real tradeoff, not a mistake to deny. Say the tradeoff out loud.

3.3 useRef

> **LIKE YOU ARE FIVE**
> A ref is a box that does not cause a redraw when you change what is inside. You use it to point at a real DOM node, or to remember a value between renders without showing a new picture.

`AuthFlowModal.tsx` holds `frameRef` and `panelRef` so it can measure the modal and scale it. Changing a ref does not re-render. If the screen must update, use state.

Interview line: "State is for values the UI shows. Refs are for values the UI does not show, or for DOM nodes."

3.4 useMemo

```
const value = useMemo(() => ({ activeStore: store, setActiveStore }), [store])
const errors = useMemo(() => { /* derive errors */ }, [step, data, region, country, city])
```

`useMemo` caches a calculation until its dependencies change. It is an optimization, not a correctness tool, except when you need a stable identity.

Two different uses in this repo:

- **StoreContext:** the context value object must stay the same reference if `store` did not change. Otherwise every context consumer re-renders when the provider's parent re-renders.
- **SignupFlow errors:** this is derived data. `useMemo` avoids rebuilding the error map. It would also be correct to compute it during render without `useMemo`. Computing during render is simpler until you measure a cost.

> **SAY THIS IN THE INTERVIEW**
> "I memoize a context value so consumers do not re-render when the provider re-renders for unrelated reasons. I do not wrap every calculation in `useMemo`. Derived data can live in the render body. I reach for `useMemo` when the calculation is expensive or when referential equality matters."

3.5 useCallback

`useCallback(fn, deps)` is `useMemo` for functions. It matters when a child is wrapped in `React.memo` and you pass the function as a prop, or when a function sits in an effect dependency array. Creating a function is cheap. Stabilizing it is for broken memoization chains, not for style.

3.6 useContext and the provider pattern

> **LIKE YOU ARE FIVE**
> Prop drilling is whispering a secret through every room. Context is pinning the secret on the hallway board. Anyone in the house can read the board. Rooms that do not care never hear the whisper.

The whole pattern in `context/StoreContext.tsx` is small enough to rebuild from memory:

1. `createContext` makes the board. The default is `undefined` so a missing provider is obvious.
2. `StoreProvider` holds the state and renders `<StoreContext.Provider value={value}>`.
3. `useStore` calls `useContext` and throws if there is no provider. That turns a silent bug into a loud one.
4. The provider writes through to `localStorage` under `seltra:active_store` so a refresh remembers the store.

The dashboard imports `useStore()`. It does not receive the store through twenty layers of props.

> **TRAP**
>
> Context is not a high-frequency store. Every consumer of that context re-renders when the value changes. A chatty value (mouse position, every keystroke of a giant tree) will hurt. For that, you split contexts, or use an external store. For "which store is open," context is the right tool.

> **SAY THIS IN THE INTERVIEW**
>
> "I lift state to the closest common parent. If that parent is far away and many layers would only forward the prop, I use context. I always ship a custom hook that throws outside the provider. I memoize the value. I do not put the entire app state in one context, because any change re-renders every subscriber."

3.7 Custom hooks

A custom hook is a function whose name starts with `use` and which may call other hooks. It exists to share behavior, not UI. `useStore` is one. You could extract `useIdleLogout` from `SessionGuard` the same way. The rules of hooks apply inside it, and it must be called from a component or another hook.

---

4. How React decides what changed

4.1 Reconciliation

When a parent re-renders, React walks the new element tree and the old one.

- Same component type in the same position: update props, keep state.
- Different type in that position: unmount the old tree, mount the new one, state is destroyed.
- Lists: match by key, not by position.

Practical trick: changing a component's `key` resets its state. `authSession` in the header is incremented so the auth modal can remount clean. If you see `key={authSession}` on a modal, that is an intentional reset, not a list key.

4.2 Batching

Several `setState` calls in one event handler become one render. You will not see a half-updated form between them. This has been true for timeouts and promises as well since React 18.

4.3 Strictness about children

Do not put hooks after an early `return` that sometimes runs. Do not create components inside components.

```
function Page() {
  function Child() { const [n, setN] = useState(0); return <button onClick={() => setN(n + 1)}>{n}</button> }
  return <Child />
}
```

That inner `Child` is a new function every render, so React sees a new component type and resets state. Define `Child` outside, or make it a plain render helper that is not used as `<Child />`.

4.4 forwardRef, displayName, and composition

`components/ui/button.tsx` is an interview favorite because it is small and dense.

```
const Button = React.forwardRef<HTMLButtonElement, ButtonProps>(
  ({ className, variant, size, asChild = false, ...props }, ref) => {
    const Comp = asChild ? Slot : "button"
    return <Comp className={cn(buttonVariants({ variant, size, className }))} ref={ref} {...props} />
  },
)
```

- `forwardRef` lets a parent point a ref at the real DOM button inside.
- `VariantProps` comes from `class-variance-authority`. `variant` and `size` pick Tailwind class strings. The component does not use a pile of if statements for styles.
- `asChild` uses Radix `Slot`, which merges the button's props onto the child element. `<Button asChild><Link href="/auth">Log in</Link></Button>` renders one link, not a button wrapped around a link. Nested interactive elements are an accessibility bug.
- `...props` forwards `onClick`, `disabled`, `type`, and `aria-*`.

> **SAY THIS IN THE INTERVIEW**
>
> "Our button is a styled primitive. Variants are data, not branches. `asChild` is composition: the button look can sit on a link without invalid HTML. Refs are forwarded so parents and libraries can focus the node. Class names go through `cn`, which is `clsx` plus `tailwind-merge`, so later Tailwind classes override earlier ones instead of both applying and fighting."

---

5. Next.js: the printing press

> **LIKE YOU ARE FIVE**
>
> React knows how to draw a component. It does not know that `/` is the homepage and `/store/de-yogo` is a shop. Next.js looks at the `app` folder and builds that map for you. It can also draw the first picture in the back room, so the visitor receives HTML immediately, which is good for speed and for Google.



Figure 5. A folder in `app` is a piece of the URL. The `page.tsx` inside it is the screen that comes out.



Figure 6. The kitchen is the server: it can prepare the page. The table is the browser: clicks and typing happen there.

5.1 App Router file conventions

File	Job in Seltra
`app/layout.tsx`	Root frame: fonts, theme, providers, toaster. Wraps every page.
`app/page.tsx`	The `/` route. Exports metadata and renders `HomePage`.
`app/features/page.tsx`	The `/features` route. A folder name is a URL segment.
`app/store/[slug]/page.tsx`	`/store/anything`. `anything` arrives as `slug`.
`app/store/[slug]/order/[status]/page.tsx`	Two dynamic segments.
`app/(dashboard)/dashboard/page.tsx`	URL is `/dashboard`. The parentheses are a route group, not a segment.
`app/(dashboard)/layout.tsx`	A layout only for routes inside that group. This one sets no-index metadata.
`app/not-found.tsx`	The 404 screen.
`app/api/health/route.ts`	A server endpoint, not a page. `GET` returns JSON.
`app/sitemap.ts`, `robots.ts`, `manifest.ts`	SEO and PWA metadata generated by code.

There is no separate router table. The file tree is the router. Interviewers call this file-system routing.

5.2 Layouts do not remount on navigation

> **LIKE YOU ARE FIVE**
> The picture frame stays on the wall. Only the photo inside changes. That is why the theme and the session guard survive when you go from `/` to `/features`.

Layouts wrap child layouts and pages. The root layout must include `<html>` and `<body>`. Nested layouts must not.

A navigation with `<Link>` from `next/link` swaps the page without a full browser reload. A plain `<a href>` reloads everything. Use `Link` for your own pages.

5.3 Server Components and Client Components

This is the question that separates people who have used Next.js from people who have read the docs once.

> **LIKE YOU ARE FIVE**
> Some pieces are built in the kitchen and only the finished plate is carried out. Those are Server Components. They can read secrets and talk to the database, and their JavaScript is not shipped to the phone. Other pieces must still wiggle on the table: buttons, typing, animations. Those are Client Components. The `'use client'` line is a sticker that says “send this file to the browser.”

	Server Component	Client Component
Default in `app/`	Yes	Only with `'use client'` at the top
Can be `async` and `await fetch`	Yes	No. Fetch inside effects or event handlers.
`useState`, `useEffect`, browser APIs	No	Yes
Reads secret env vars	Yes	Only `NEXT_PUBLIC_` variables, which are public
Exports `metadata`	Yes	No

IN THIS PROJECT

`app/page.tsx` has no `'use client'`. It exports `metadata` and renders `<HomePage />`. That file is a Server Component. `components/marketing/HomePage.tsx` begins with `'use client'` because the homepage tracks input, scrolling, and refs. The server page is a thin shell. The interactive island is the client child.

`app/store/[slug]/page.tsx` is an async Server Component. It awaits the store, then passes plain data into `StorefrontCanvas`. Data passed from server to client must be serializable: objects, arrays, strings, numbers. You cannot pass functions, class instances, or dates you expect to stay as `Date` objects without converting them.

`app/(dashboard)/dashboard/page.tsx` is `'use client'` from the first line. The whole dashboard is a client island. That is why it reads `NEXT_PUBLIC_API_BASE_URL` and `localStorage`.

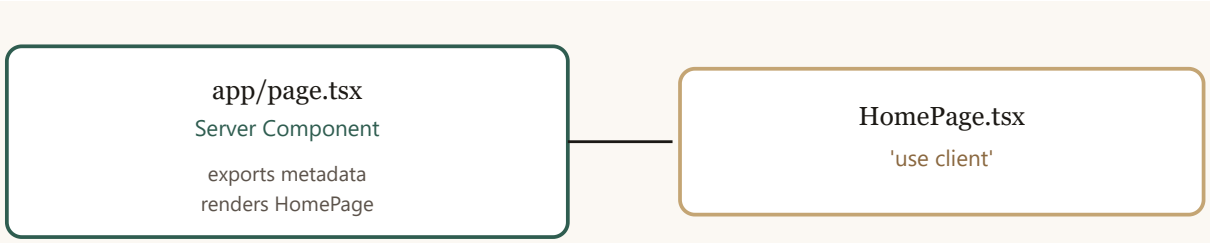


Figure 8. The URL stays a server page. The interactive homepage is a client island inside it.

SAY THIS IN THE INTERVIEW

"In the App Router, components are Server Components unless a file has `'use client'`. That directive marks a client boundary: the file and the modules it imports are included in the browser bundle. I keep pages on the server when they mostly fetch and render, and I push `'use client'` down to the smallest interactive leaf. The homepage does this. The dashboard does not, because almost all of it is interactive, and I would name that as a splitting opportunity."

TRAP

`'use client'` does not mean "this runs only in the browser." Client components are still pre-rendered to HTML on the server, then hydrated in the browser. They cannot use `useState` during a server-only thinking step, but their first render must succeed on the server. That is why touching `window` in the render body crashes, and why `StoreContext` guards `localStorage` with `typeof window`.

5.4 Hydration

LIKE YOU ARE FIVE

The server sends a paper picture. Hydration is the moment the picture becomes a toy you can touch. React walks the paper and attaches the click handlers. If the paper does not match the toy's first pose, React complains.

The root layout sets `suppressHydrationWarning` on `<html>` and `<body>`. `next-themes` adds a class such as `dark` from `localStorage`. The server cannot see that storage, so the class can differ for one moment. The attribute tells React to ignore that known mismatch on those two tags. It is not a way to hide real bugs deeper in the tree.

SAY THIS IN THE INTERVIEW

"Hydration mismatches happen when the server HTML and the client's first render disagree. Common causes are `Date.now()`, `Math.random()`, locale formatting, and reading browser-only storage during render. I keep the first render deterministic. For theme, I accept a suppressed warning on the html element because the theme class is applied by a client library after it reads storage."

5.5 Dynamic routes and async params

```
type StoreRouteProps = { params: Promise<{ slug: string }> }

export default async function StorefrontPage({ params }: StoreRouteProps) {
  const { slug } = await params
  const store = (await getStore(slug)) ?? fallback(slug)
  // ...
}
```

In Next.js 15 and in this Next.js 16 app, `params` is a Promise. You await it. Older tutorials show `props.params.slug` as a plain object. If you write that here, it will not work. Say the version when you answer.

`generateMetadata` on the same page awaits the same params, fetches the store, and builds the title, description, Open Graph image, and canonical URL from real store data. That is how a shared link to a shop shows the shop's name instead of a generic "Seltra" title.

5.6 Data fetching and revalidation

```
const res = await fetch(`${API_BASE}/api/v1/seltra/store/${encodeURIComponent(slug)}`, {
  next: { revalidate: 30 },
})
```

> **LIKE YOU ARE FIVE**
>
> The first visitor asks the kitchen for the menu. Next.js photocopies the menu for 30 seconds and hands out the photocopy. After 30 seconds the next visitor gets a fresh menu, and the photocopy is replaced. Shoppers in those 30 seconds all see the same fast copy.

Names interviewers use:

- **SSR:** render on the server for that request. In the App Router this is dynamic rendering.
- **SSG:** render at build time and reuse forever, until you rebuild.
- **ISR:** incremental static regeneration. Serve a cached page, refresh it in the background on an interval. `revalidate: 30` is that interval in seconds.
- **Client fetch:** the browser asks after the page is already showing. The dashboard does a lot of this.

`encodeURIComponent(slug)` makes a slug safe to put in a URL. `getStore` returns `null` on failure, and the page substitutes `fallback(slug)` demo products. That is a product choice: the page still renders. In an interview, say you would not show fake products on a real customer store without a clear "preview" label, because it can look like a successful load of the wrong catalog.

`API_BASE` here is `process.env.API_BASE_URL`. It is not prefixed with `NEXT_PUBLIC_`. It is only read in a Server Component, so it is not bundled to the browser. Compare with signup:

```
const API_BASE = process.env.NEXT_PUBLIC_API_BASE_URL ?? "http://localhost:3001"
```

> **SAY THIS IN THE INTERVIEW**
>
> "`NEXT_PUBLIC_` variables are inlined into client JavaScript and are public. Anything secret — service keys, private API hosts you do not want exposed — stays unprefixed and is read only on the server. I treat the public anon key as public by design, and I rely on database rules for real protection. I never put a service-role key in a `NEXT_PUBLIC_` variable."

5.7 Suspense and loading

The store page wraps `StorefrontCanvas` in `<Suspense fallback={<Skeleton />}>`. Suspense shows the fallback only if a child actually suspends: a lazy component, or a component that throws a promise while it loads. The `fetch` in this page is already awaited before that JSX runs, so the skeleton is not the loading state for `getStore`. The visitor waits on the server response, then receives HTML.

The idiomatic Next.js tool for that wait is `loading.tsx` beside `page.tsx`. Next wraps the page in Suspense for you and shows `loading.tsx` immediately while the async page resolves. There is a `not-found.tsx`. A matching `loading.tsx` and `error.tsx` would be the next files to add.

> **SAY THIS IN THE INTERVIEW**
>
> "`loading.tsx` is a route-level Suspense boundary. `error.tsx` is an error boundary for that segment and must be a client component. `not-found.tsx` renders when we call `notFound()` or when no route matches. Suspense in the tree is for nested waits. Awaiting fetch in the page and also wrapping a non-suspending child does not create a loading state by itself."

5.8 Route handlers

```
export function GET() {
  return NextResponse.json({ ok: true, service: "seltra-web", time: new Date().toISOString() })
}
```

`app/api/health/route.ts` is a backend endpoint inside the Next server. Export functions named `GET`, `POST`, and so on. This one is a heartbeat. The real business API still lives in the Nest backend. Next route handlers are the right place for BFF-style glue: hide a secret, shape a response, set a cookie. They are not automatically better than calling the backend from a Server Component.

5.9 Navigation hooks

Import from `next/navigation`, not `next/router`. The second one is the old Pages Router.

- `useRouter()` in `SessionGuard` calls `router.replace(...)` so a logout does not leave `/dashboard` in the back button as a page you can return to.
- `usePathname()` tells the guard the current path so `/auth` does not redirect to itself.
- `redirect()` from `next/navigation` is for Server Components. It throws, by design, so Next can catch it. Do not catch it and ignore it.

5.10 Metadata, fonts, images, redirects

Metadata. The root layout exports a `metadata` object: title template `"%s | Seltra"`, description, Open Graph, Twitter, icons, robots. Child pages export their own `metadata` or `generateMetadata`. The dashboard group spreads `noIndex` so private screens ask Google not to index them.

Fonts. `next/font/google` downloads Fraunces and Poppins at build time and exposes them as CSS variables on `<html>`. Geist is loaded from the `geist` package the same way. Self-hosting avoids a flash of invisible text and a third-party request during page view.

Images. `next.config.ts` lists `images.remotePatterns`. `next/image` refuses remote hosts that are not listed. That limits abuse of the image optimizer as an open proxy. `dangerouslyAllowSVG` is on; in an interview you mention SVG can carry script if the source is not trusted, so this should stay paired with hosts you control.

Redirects. `/store` redirects to `/dashboard` with `permanent: false`, a temporary redirect. Permanent redirects are cached hard by browsers. Use them only when the move is forever.

Script. The layout uses `next/script` for JSON-LD structured data so search engines can read organization and website facts. `dangerouslySetInnerHTML` is acceptable here because the JSON is produced by `JSON.stringify` from your own constants, not from a stranger's HTML.

5.11 The request, end to end

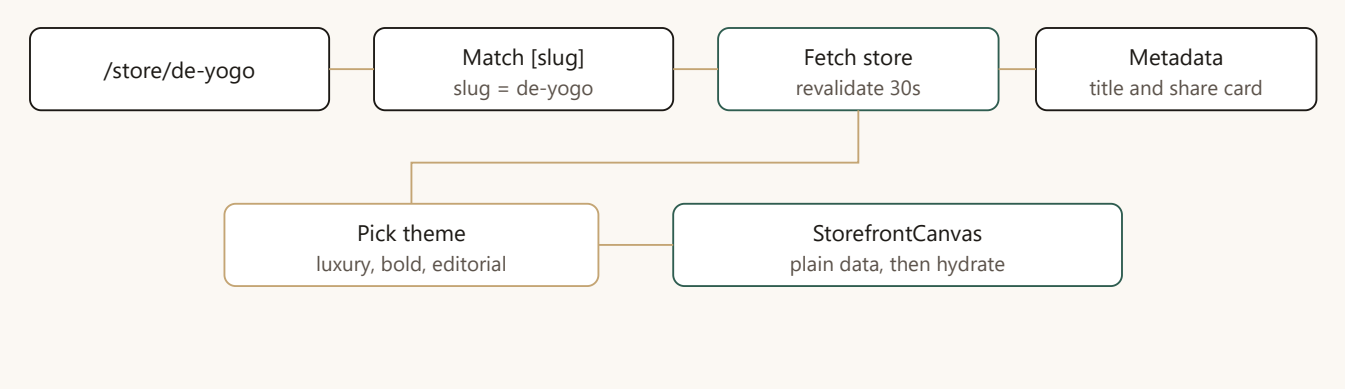


Figure 9. One visit to a public shop. The server fetches, picks a theme, and sends HTML. The browser hydrates the parts that can be clicked.

Someone opens `/store/de-yogo`.

1. Next matches `app/store/[slug]/page.tsx`. `slug` is `de-yogo`.
2. The root layout renders the frame: fonts, theme provider, store provider, session guard.
3. `generateMetadata` fetches the store and sets the tab title and social cards.
4. The page function fetches again (Next can dedupe identical fetches in one render pass) with a 30-second revalidate.
5. If the API fails, a fallback object is built from the slug so the screen is not blank.
6. Theme key is chosen from brand personality and layout variant: luxury, bold-dark, editorial, or minimal-light.
7. Plain store data is passed into the canvas, which is the visible shop.
8. The browser hydrates client components inside that tree so buttons work.

Someone opens `/dashboard` while logged out.

1. The group layout applies no-index metadata. The page is a client component.
2. `SessionGuard`'s effect sees there is no token and returns early on paths that are not protected inside the effect itself. Read that file carefully before you claim every route is guarded: the effect returns immediately when there is no token, and it skips work on `/auth`. Protection is "if a token exists, watch it and log it out." A page that assumes a user is present must still check the token itself and redirect. Know this limitation. It is a strong interview observation.

---

6. Forms, validation, and trust

> **LIKE YOU ARE FIVE**
>
> The form is a clipboard. React holds the answers. Before you send the clipboard to the office, you check the handwriting. You check it again in the office, because anyone can skip your website and send a messy clipboard directly.

`SignupFlow.tsx` is the form lesson.

- A Zod schema names every field, its type, and its limits. `z.infer<typeof schema>` creates the TypeScript type from the schema so the two cannot drift.
- UI errors for the current step are derived in `useMemo`.
- `attempted` controls whether required-field errors show. Format errors, like a bad email, can show earlier so the user is not punished for still typing.
- `goNext` refuses to advance when `errors` is not empty, and sets `attempted` so the messages appear.
- `submit` runs `schema.safeParse`. On failure it toasts the first issue. On success it posts JSON to the backend.
- A toast is a small message that does not navigate away. The project uses Sonner.

> **SAY THIS IN THE INTERVIEW**
>
> "Client validation is for fast feedback. Server validation is for truth, because the client can be modified. I like a single schema shared by both sides when the stack allows it. Zod's `safeParse` returns a result instead of throwing, which is easier to turn into field errors. I keep derived errors out of state."

The project also depends on `react-hook-form` and `@hookform/resolvers`. That library is the industrial version of what `SignupFlow` does by hand: it tracks fields, touched state, and validation without re-rendering the whole tree on every keystroke. If an interviewer asks "why not hand-roll," a fair answer is: hand-rolled is clear for a short wizard; React Hook Form pays off when there are many fields, third-party inputs, and you want uncontrolled registration for performance.

---

7. Auth, in plain language

> **LIKE YOU ARE FIVE**
>
> A token is a stamped ticket. The browser keeps the ticket. Every request to the office shows the ticket. If the ticket is old, or the person has been quiet for an hour, the guard takes the ticket and walks them back to the login door.



Figure 7. The ticket is the access token. SessionGuard watches the ticket and the clock. The real lock is still the API, not the picture on the door.

What the code actually does:

- `lib/supabase.ts` builds one browser Supabase client and reuses it. Creating a client per render would be wasteful and would break the auth subscription.
- The client persists the Supabase session, refreshes tokens, and detects a session in the URL after a magic-link redirect.
- `storeSeltraSession` also copies the access token and user JSON into `localStorage` under `seltra:token` and `seltra:user`.
- `SessionGuard` listens with `onAuthStateChange`. A new access token is written to `localStorage`. A sign-out clears Seltra keys and sends the user to `/auth?next=...`.
- Idle timeout is one hour from the last pointer, key, touch, or scroll. Token expiry is read by base64-decoding the JWT payload and reading `exp`. Logout happens at whichever comes first.
- A `storage` listener logs this tab out if another tab deletes the token.

TRAP YOU SHOULD VOLUNTEER

Decoding a JWT in the browser does not verify its signature. Anyone can edit `localStorage` and invent an `exp`. Expiry here is only a user-experience timer. The backend must verify the token on every protected request. Also, tokens in `localStorage` are readable by any script on the page, so an XSS bug can steal them. The stronger design is an httpOnly, Secure, SameSite cookie that JavaScript cannot read. I would say that tradeoff without being asked.

> **SAY THIS IN THE INTERVIEW**
>
> "The client stores a Supabase access token and mirrors it for API calls. A guard component subscribes to auth changes, enforces an idle timeout, and clears storage on sign-out. I do not trust the client's view of expiry. Authorization is enforced on the server. If I were hardening this, I would move the session to an httpOnly cookie and keep the client guard as a UX layer only."

---

8. Styling without losing the plot

Tailwind utilities live on the `className` string. `cn()` in `lib/utils.ts` joins them and lets the later conflicting utility win.

Design tokens such as `bg-primary` and `text-foreground` point at CSS variables. The toaster in the root layout reads those variables so notifications match the theme. `next-themes` toggles a class on `<html>`. Dark is the default. System theme is off, and the storage key is `seltra-theme`.

You do not need to recite Tailwind in an interview. You need to say: styles are colocated with components, variants are generated, conflicts are merged, and theme is a class plus CSS variables.

---

9. TypeScript, only what you use

- `type` and `interface` describe shapes. Props are types so a missing `slug` is a compile error, not a production bug.
- `string | null` means the value might be missing. You must handle both. That is why photos in the signup flow are `string | null`.
- Generics show up as `useState<StoreData | null>` and `set<K extends keyof typeof data>`. The second one means "the key must be a real field, and the value must match that field." You can describe it as a locked pair: you cannot set `email` to a number.
- `z.infer` derives a type from a runtime schema. One source of truth.
- `strict: true` is on. Do not silence errors with `any` unless you are at a boundary you do not control, and then narrow it immediately.

> **SAY THIS IN THE INTERVIEW**
>
> "TypeScript does not run in the browser. It disappears at compile time. Runtime checks still need Zod or manual guards. I use types for developer feedback and schemas for data that crossed a network."

---

10. Performance stories you can tell honestly

- **Server-render the public shop.** Less JavaScript, faster first paint, metadata for sharing. The store route does this.
- **Cache with `revalidate`.** Avoid hitting the API on every view.

- **Push client boundaries down.** The homepage shell is a server page. The dashboard is not. A senior answer names the cost: a multi-thousand-line client page ships a large bundle and cannot keep secrets.
- **Stable context values.** `useMemo` in `StoreProvider`.
- **`next/image` and `next/font`.** Sized images and self-hosted fonts reduce layout shift.
- **Passive scroll listeners.** `{ passive: true }` tells the browser the listener will not call `preventDefault`, so scrolling stays smooth.
- **Do not memoize by superstition.** Measure first. React re-rendering a small component is cheap. Re-rendering a 4,000-line dashboard on each keystroke is not. The fix is splitting components so state lives next to the input that changes.

> **SAY THIS IN THE INTERVIEW**
>
> "If a keystroke re-renders a huge tree, I move that state down into the input's component, or split the page so the expensive subtree receives stable props and can be memoized. I do not start by wrapping every function in `useCallback`."

---

11. Build Seltra again, in small pieces

Do these in order. Each one is allowed to be ugly. Each one must run.

1. **A component and props.** A `ProductCard` that receives `name` and `price` and returns a figure. Render three of them from an array with stable keys.
2. **State.** A counter button. Then a controlled search box that filters the products.
3. **Lifting state.** A parent holds `query`. A child input calls `onQueryChange`. A sibling list receives the filtered array.
4. **Effect and cleanup.** A header that sets `scrolled` from `window.scrollY` and removes the listener on unmount.
5. **Context.** Recreate `StoreProvider` and `useStore` with the throw-if-missing hook. Persist to `localStorage` with a window guard.
6. **Routes.** `app/page.tsx`, `app/features/page.tsx`, `app/store/[slug]/page.tsx`. Log the slug. Add a root layout that wraps children in your provider.
7. **Server fetch.** Make the store page async. `fetch` a mock endpoint with `revalidate: 30`. Handle failure with a typed empty state, not a fake catalog, unless the UI says "preview."
8. **Client island.** A small `'use client'` add-to-bag button that receives a serializable product.
9. **Metadata.** Static metadata on the home page. `generateMetadata` on the store page.
0. **Form.** A two-step wizard. Derived errors. Zod `safeParse` on submit. Toast on failure.
1. **Button primitive.** Variants with `cva`, `cn`, and `asChild` so a link can wear the button styles.
2. **Session sketch.** A fake token in memory. A guard that redirects with `useRouter` from `next/navigation` and cleans up a timer.

When you can do item 11 without opening this file, you can build the front of this application. The rest of Seltra is these patterns repeated: more fields, more routes, more client islands.

12. A 14-day study plan

13. Interview bank

Speak the answer, then add one Seltra detail. Stop. Do not lecture.

React fundamentals

1. What is React?

> **ANSWER**
> A library for building UI out of components. I describe the screen as a function of state. When state changes, React re-renders and updates the DOM. In Seltra, a page is a tree: layout, providers, then a page component such as the storefront or the dashboard.

2. What is the difference between props and state?

> **ANSWER**
> Props come from the parent and are read-only. State is owned by the component and changed with a setter, which schedules a render. `SignupFlow` receives photos as props and owns the wizard step as state.

3. Why can’t a child change a prop?

> **ANSWER**
> One-way data flow. If children mutated props, any component could change shared data and the UI would be impossible to trace. The child calls a callback such as `setActiveStore`, and the owner updates state.

4. What does rendering mean? Is it the same as painting pixels?

> **ANSWER**
> Rendering is running the component to produce a React element tree. Commit is when React applies changes to the DOM. The browser then paints. I keep render pure.

5. Why are keys required in lists? Why not the index?

> **ANSWER**
> Keys let React match items across renders. Index keys tie identity to position, so reordering can reuse the wrong component state, especially inputs. I use a stable id, like a product id.

6. What happens if you call a hook inside an if?

ANSWER

Hooks rely on call order. A conditional hook shifts the list and state gets assigned to the wrong hook. I call hooks unconditionally at the top.

7. When do you use useEffect?

ANSWER

To synchronize with an external system after paint, with a cleanup. SessionGuard's subscriptions and timers are the example. I do not use an effect to copy props into state or to calculate errors I can derive.

8. What is a stale closure?

ANSWER

An effect or callback that captured old state because it was created on an earlier render and its dependencies were incomplete. The fix is a correct dependency array, or a functional update, or resetting with a key when identity should change.

9. useMemo versus useCallback versus React.memo?

ANSWER

`useMemo` caches a value. `useCallback` caches a function. `React.memo` skips a child render when its props are shallowly equal. They work together. StoreContext memoizes its value so the object identity is stable. I do not add them everywhere.

10. What is prop drilling and what do you do about it?

ANSWER

Passing data through components that do not use it, only so a grandchild can. I lift state to the closest owner first. If the gap is wide, I use context, the way `useStore` shares the active store. If the data updates many times a second, I avoid a single broad context.

11. Controlled versus uncontrolled inputs?

ANSWER

Controlled: React state is the source of truth, `value` plus `onChange`. Uncontrolled: the DOM holds it, read via ref on submit. Seltra's signup fields are controlled because each step validates them.

12. How do you reset a form or a modal's state?

ANSWER

Set state back to initial values, or change the component `key` so React remounts it and runs initial state again. Remounting is the blunt, reliable reset.

13. What is children? What is composition?

ANSWER

`children` is a prop for nested JSX. Composition means building behavior by wrapping, not by inheritance. The root layout, providers, and `Button asChild` are composition. React components rarely use class inheritance for UI reuse.

14. What would cause an infinite render loop?

ANSWER

Setting state during render unconditionally, or an effect that sets state and depends on that state without a condition. Example: an effect with `[store]` that always calls `setStore` to a new object. I update state in handlers, or in effects only when the next value is actually different.

15. How do error boundaries fit?

ANSWER

They catch render errors in children and show a fallback. They do not catch event-handler errors, async errors, or errors in the boundary itself. In the App Router, `error.tsx` is that boundary for a route segment and must be a client component. I still use try/catch around `fetch`, which is what `getStore` does.

Next.js

16. What is the App Router?

ANSWER

Routing from the `app` directory. `page.tsx` makes a route, `layout.tsx` wraps segments and preserves state across navigations, `loading.tsx` and `error.tsx` are special UI, `route.ts` is an HTTP handler. Seltra uses this, not the Pages Router.

17. What does 'use client' do?

ANSWER

It marks a module and its imports as a client boundary so they can use state, effects, and browser APIs. It is still server-rendered to HTML, then hydrated. I put it as low in the tree as I can. The home route is a server page that renders a client `HomePage`.

18. What can you pass from a server component to a client component?

> **ANSWER**
>
> Serializable props: plain objects, arrays, strings, numbers, booleans. Not functions, not class instances. The store page passes the fetched store object into the canvas. If I need behavior, the client component defines the function itself.

19. Explain SSR, SSG, and ISR with this codebase.

> **ANSWER**
>
> The store page fetches on the server and can be cached. `next: { revalidate: 30 }` is time-based revalidation, the ISR idea: serve the cached result, then refresh at the interval. A force-dynamic page would render per request. Client pages like the dashboard render their shell and then fetch in the browser.

20. Why is params a Promise here?

> **ANSWER**
>
> Starting in Next.js 15, and in this Next 16 app, `params` and `searchParams` are async so the framework can stream and avoid synchronous blocking. I `await params` in both the page and `generateMetadata`. I mention the version so I do not mix it up with Next 14 tutorials.

21. What is a route group?

> **ANSWER**
>
> A folder wrapped in parentheses, like `(dashboard)`, organizes files and can have its own layout without adding a URL segment. `/dashboard` is the URL. That layout attaches no-index metadata so private pages are not SEO targets.

22. Link versus anchor versus router.replace?

> **ANSWER**
>
> `Link` prefetches and does client-side transitions, preserving layouts. A raw anchor does a full document load, which I still want for external URLs or when I need a full reset. `router.replace` navigates without adding a history entry. Logout uses replace so Back does not return to a private screen as easily. I import `useRouter` from `next/navigation`.

23. Where do you set the document title?

ANSWER

The Metadata API. Static `export const metadata` on server pages, `generateMetadata` when it depends on data, as on the store slug. The root title template is `%s | Seltra`. Client components should not own SEO metadata.

24. How does caching interact with fetch?

ANSWER

In the App Router, `fetch` can participate in Next's data cache. `revalidate` sets a lifetime. `cache: 'no-store'` opts out when data must be fresh per user. I would not cache a personalized dashboard response the way I cache a public storefront. Identical fetches in one server render can be deduped.

25. What is hydration, and why might this app suppress a warning?

ANSWER

Hydration attaches event handlers to server HTML. A mismatch means the first client render differed. `suppressHydrationWarning` on html and body covers the theme class from `next-themes`, which depends on stored preference the server cannot know. I would not suppress warnings on arbitrary divs.

26. How do environment variables work?

ANSWER

Unprefixed variables are server-only. `NEXT_PUBLIC_` is exposed to the browser. The store page reads `API_BASE_URL` on the server. Client forms read `NEXT_PUBLIC_API_BASE_URL`. The Supabase anon key is public; row-level security has to be the real lock.

27. What does next/image buy you, and what is remotePatterns?

ANSWER

Optimization, sizing, and lazy loading, which helps layout stability. `remotePatterns` is an allowlist so the optimizer cannot be pointed at arbitrary hosts. Our config allows known image CDNs and a few asset hosts.

28. What is a route handler for, if you already have a backend?

ANSWER

Small server endpoints next to the UI: health checks, webhooks, or a backend-for-frontend that hides secrets and shapes data. `app/api/health/route.ts` is the simple case. Domain logic for stores and applications stays in the Nest API.

Applied to Seltra

29. Walk me through the store page.

> **ANSWER**
>
> It is an async server component. It awaits `params.slug`, fetches `/api/v1/seltra/store/:slug` with a 30-second revalidate, falls back if the request fails, picks a theme from brand personality and layout variant, and renders `StorefrontCanvas` inside Suspense. `generateMetadata` uses the same fetch to set social tags and a canonical URL on the store subdomain.

30. How does the signup wizard manage correctness?

> **ANSWER**
>
> Field state lives in one object updated immutably. Step errors are derived. Next is blocked when the current step has errors. Submit runs a Zod schema with `safeParse` and only then posts. I would repeat that schema check on the server.

31. How does logout work?

> **ANSWER**
>
> `SessionGuard` signs out of Supabase, clears Seltra localStorage keys, and `replace`s to `/auth` with a `next` query so login can return the user. It also logs out at token expiry or after an hour idle, and if another tab removes the token.

32. Why memoize the context value?

> **ANSWER**
>
> So the provider can re-render without changing the value identity when the store is unchanged. Context consumers re-render when the value fails `Object.is`. A fresh object literal every render would defeat that.

33. What is wrong, or risky, in the current frontend?

> **ANSWER**
>
> I would mention four things. The access token in localStorage is exposed to XSS; httpOnly cookies are stronger, and client-side JWT decode is not verification. The dashboard file is thousands of lines, so state updates are hard to isolate. The store fallback can present demo products when the API fails, which should be labeled as a preview. SessionGuard does not by itself redirect anonymous users away from private pages; each private view still needs an explicit check. I would say these as improvements, calmly, after showing I understand why the code is shaped this way.

34. How would you test a component like the button or the wizard?

ANSWER

React Testing Library: render the button, assert the variant class behavior through its accessible role, and click it. For the wizard, type invalid input, assert the error appears only when appropriate, click next, and mock `fetch` to assert the payload. I test behavior the user sees, not private state.

35. How would you add a new public page?

ANSWER

Add `app/pricing/page.tsx` as a server component, export metadata, and link to it with `next/link`. If a small part needs state, extract that part into a client component. I would not mark the whole page as client just to use one `useState`.

---

14. Senior notes: how you sound like you have shipped

- **Name the boundary.** "This is a server component up to here, and a client island from here." Draw it with your hand if you are at a whiteboard.
- **Separate UX auth from real auth.** Guards and idle timers are experience. The API is the lock.
- **Prefer derived state.** Errors, filtered lists, theme keys. If it is a formula, do not store the result.
- **Colocate state.** The component that changes most often should own the state that changes. That is the performance plan for the dashboard.
- **Be precise about versions.** React 18.3, Next.js 16, async params, App Router. Interviewers lose trust when versions are mixed.
- **Talk about failure.** `getStore` catches and returns null. Say what the user should see, and what you would log.
- **Accessibility in one sentence.** Buttons that navigate should sometimes be links. `asChild` exists so we do not put a link inside a button. Labels point at inputs with `htmlFor`. Icon-only buttons need an accessible name.
- **Do not pretend the dashboard is a template.** It is a large client page. Your judgment about splitting it is more impressive than defending it.

15. Glossary

Day	Read	Do	Say out loud
1	Chapters 0–1	Draw the folder map from memory	What is a component?
2	Chapter 2	Practice items 1–3	Props vs state
3	Chapter 3.1–3.3	Practice item 4	What belongs in an effect?
4	Chapter 3.4–3.7 and `StoreContext.tsx`	Practice item 5	Why context, why not always?
5	Chapter 4 and `button.tsx`	Practice item 11	Keys, and what `asChild` prevents
6	Chapter 5.1–5.4	Practice item 6	Server vs client component
7	Chapter 5.5–5.8 and the store page	Practice items 7–9	What does `revalidate: 30` mean?
8	Chapter 6 and `SignupFlow.tsx`	Practice item 10	Why validate twice?
9	Chapter 7 and `SessionGuard.tsx`	Practice item 12	Where must auth really be enforced?
10	Chapters 8–10	Split one big component into two on paper	How would you speed up a huge client page?
11	Interview bank, questions 1–15	Answer with a timer, 90 seconds	Record yourself once
12	Questions 16–30	Same	Point at a file for each answer
13	Chapter 14, the senior notes	Write five improvements	Lead with the cookie and the giant dashboard
14	Rebuild the store route from a blank app	No peeking for 45 minutes, then diff	Walk the request end to end

Word	Plain meaning
Component	A function that returns UI.
JSX	HTML-like syntax inside JavaScript.
Props	Inputs from the parent.
State	Memory that causes a redraw when it changes.
Render	Calling the function to describe the UI.
Commit	Writing the description into the DOM.
Hydration	Waking server HTML up so it can handle events.
Hook	A function like `useState` that taps React features. Call order matters.
Effect	After-paint work with the outside world, plus cleanup.
Ref	A mutable box that does not re-render, often a DOM node.
Context	A way to share a value with a subtree.
Key	A stable identity for an item in a list.
Client component	A module in the browser bundle. Marked with `'use client'`.
Server component	A module that runs on the server and is not shipped as that component's logic.
Layout	UI that wraps a route segment and stays mounted.
Route group	A `(folder)` that does not affect the URL.
Dynamic segment	A `[folder]` that captures part of the URL.
Revalidate	How long a cached server fetch may be reused.
Metadata	Title, description, and social cards for a route.
Hydration mismatch	Server HTML and the first client render disagreed.
Serializable	Data that can be turned into JSON and passed to a client component.
BFF	A thin server endpoint beside the UI that fronts another API.

16. One-page cheat sheet

```
Component     function Name(props) { return <jsx/> }     capital name, pure render
Props         read-only inputs from parent               data down
State         useState, setter schedules render          memory
Update        setX(v) or setX(prev => next)              copy objects, do not mutate
Derived       compute during render                      do not mirror state
List          items.map(item => <Row key={item.id}/>)    stable ids
Effect        useEffect(() => { ...; return cleanup }, [deps])
Ref           useRef, no re-render                       DOM or instance values
Context       createContext + Provider + hook that throws
Client        'use client' at top of file                hooks and browser
Server        default in app/                            async, secrets, metadata
Route         app/segment/page.tsx                       layout.tsx wraps it
Dynamic       app/store/[slug]/page.tsx                  await params
Cache         fetch(url, { next: { revalidate: 30 } })
Public env    NEXT_PUBLIC_*                              everything else is server-only
Navigate      <Link href>   useRouter from next/navigation
Validate      Zod on the client and again on the server
Auth UX       guard, idle timer, storage                 real check is the API
```

17. The shortest true story

If you forget everything except this, you can rebuild the idea of the app.

1. A screen is a function of data.
2. Parents pass data down. Children send events up.
3. If the screen must remember a click, that memory is state.
4. If many screens must share memory, lift it, then use context when the lift is too far.
5. After the picture is drawn, effects talk to the browser, and they clean up after themselves.
6. In Next.js, a file in `app` is a URL.
7. The server draws what it can. The browser only receives the parts that must move.
8. Public pages fetch on the server and can be cached. Private, clicky pages are client islands and must not be trusted with secrets.
9. Check untrusted input twice. Check identity on the server.
0. When a file gets huge, split it along the state that changes.

Open `app/layout.tsx`, `app/page.tsx`, `app/store/[slug]/page.tsx`, `context/StoreContext.tsx`, `components/ui/button.tsx`, `components/marketing/SignupFlow.tsx`, and `components/auth/SessionGuard.tsx`. Those seven files are the course. This document is the commentary.

Written from the Seltra frontend as it exists in this repository: React 18.3, Next.js 16 App Router, TypeScript. When a tutorial on the internet disagrees about `params` or `useRouter`, prefer this codebase's version and say so.